

STEP

GitHub Repository Structure Guide

Semester 3 — Version Control & Daily Workflow Standard

Repository Name

Step_semester_3

This document explains, step by step, how to set up a GitHub account and organize the “Step_semester_3” repository using a clean branching model: a Main branch for documentation, a Develop branch for the base project, and Feature branches for daily session work.

Overview

This guide walks through the complete GitHub workflow that must be followed for the Step Semester 3 coursework. It covers four parts:

1. Creating a GitHub account
2. Creating the Step_semester_3 repository
3. Setting up the branching structure (Main, Develop, and Feature branches)
4. Organizing the package structure inside each feature branch

Why this structure matters

Following a consistent branching and folder structure mirrors real-world software development practices, makes daily progress traceable through the README, and keeps every session's work isolated and easy to review.

Step 1: Create a GitHub Account

If you do not already have a GitHub account, follow these steps:

5. Go to <https://github.com> in your web browser.
6. Click Sign up in the top-right corner of the page.
7. Enter your email address, create a password, and choose a unique username.
8. Verify your email address using the confirmation link or code sent by GitHub.
9. Complete the account setup by answering the short preference questions (or skip them).
10. Once verified, you will land on your GitHub dashboard — your account is ready to use.

Tip

Use a professional username (for example, based on your real name) since this account may be shared with mentors, reviewers, or future employers.

Step 2: Create the Repository

After logging in, create the repository that will hold all your Semester 3 work:

11. Click the “+” icon in the top-right corner and select New repository.
12. Set the Repository name to: Step_semester_3
13. Choose Public or Private, based on your mentor's instructions.

14. Check “Add a README file” so the Main branch is initialized with a README.md.
15. Click Create repository.

Step 3: Set Up the Branching Structure

The repository uses three types of branches, each with a specific, fixed purpose. Do not mix content between branches.

Branch	Purpose	Content Allowed
main	Documentation & daily progress log	Only README.md
develop	Base/empty project skeleton	Only the empty IDE project structure
feature/session_n	Actual coding work per session	Package with solved problems

Create the branches in this order:

16. main — usually created automatically as the default branch.
17. develop — create this branch from main.
18. feature/session_1, feature/session_2, ... — create each of these from develop, one per session.

```
# Example commands (run locally after cloning the repo)
git checkout main
git checkout -b develop
git push -u origin develop

git checkout develop
git checkout -b feature/session_1
git push -u origin feature/session_1
```

Step 4: Main Branch — README Rules

The Main branch contains only the README.md file. It is never used for code. Update it after every session with a new dated entry using the format below:

```
## Date: DD-MM-YYYY

**Today's Work:**
- Summary of what was completed in this session
```

```
**Next Session Plan:**
- What you plan to work on next

**Issues Faced:**
- Any blockers, errors, or doubts encountered (or 'None')

---
```

Add a new dated block above the previous one each day, so the README reads as a running daily log of the whole semester.

Step 5: Develop Branch — Project Skeleton

The Develop branch holds only the empty project structure created by your IDE (e.g., IntelliJ IDEA, Eclipse) — no solved problems, no packages with logic.

19. Create a new Java project in your IDE.
20. Do not add any code, classes, or packages yet.
21. Push this empty project structure to the develop branch.

Important

Feature branches are NOT merged back into develop for this course. develop stays a clean, empty base project throughout the semester.

Step 6: Feature Branches — One per Session

Every coding session gets its own feature branch, always created from develop, never from main or another feature branch.

Session	Branch Name
Session 1	feature/session_1
Session 2	feature/session_2
Session 3	feature/session_3
...	feature/session_n

Naming rules:

- Always lowercase, using the pattern feature/session_<number>.

- Use an underscore before the number — no spaces, no capital letters.
- Branch from develop every time: `git checkout develop` then `git checkout -b feature/session_n`.

Step 7: Package Structure Inside a Feature Branch

Inside each feature branch, code is organized by topic. The topic name becomes the top-level package, and every topic package contains exactly two sub-packages:

- `class_problems` — problems solved and explained during the live session
- `assignment_problems` — problems given as take-home assignments

Example: if Session 1's topic is “String”, the package layout looks like this:

```
src/  
  main/  
    java/  
      string/  
        class_problems/  
          ReverseString.java  
          PalindromeCheck.java  
        assignment_problems/  
          AnagramCheck.java  
          CountVowelsConsonants.java
```

Final Repository Structure (Summary)

Putting it all together, the complete repository looks like this:

```
Step_semester_3  
|  
|-- main  
|   -- README.md    (dated daily log: work done, next steps, issues)  
|  
|-- develop  
|   -- empty Java project structure (no code yet)  
|  
|-- feature/session_1  
|   -- string/  
|       |-- class_problems/
```

```
|          |-- assignment_problems/
|
|-- feature/session_2
|   -- <topic_2>/
|       |-- class_problems/
|       |-- assignment_problems/
|
|-- feature/session_n
|   -- <topic_n>/
|       |-- class_problems/
|       |-- assignment_problems/
```

Quick Checklist

- GitHub account created and verified
- Repository Step_semester_3 created
- main branch has README.md only, updated after every session
- develop branch has only the empty project skeleton
- A new feature/session_n branch created from develop for each session
- Topic package created per session, with class_problems and assignment_problems sub-packages
- Feature branches are kept separate and are not merged into develop